ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
Τμήμα Ηλεκτρολόγων Μηχανικών & Τεχνολογίας Υπολογιστών

Λειτουργικά Συστήματα (ECE ΓΚ702)

Δραστηριότητα 2: Χρονοπρογραμματισμός Διεργασιών στο xv6

Φοιτητής: Παναγόπουλος Αθανάσιος
Αριθμός Μητρώου (ΑΜ): 1107645
Ημερομηνία: 7 Νοεμβρίου 2026
Repository: https://github.com/RealThanosP/thanOS-v6

1 Άσκηση 1: Αναπαράσταση διεργασίας στο xv6

Για κάθε διεργασία το kernel κρατά μία εγγραφή τύπου `struct proc` στον πίνακα `ptable.proc`. Βρίσκουμε τον ορισμό στο `include/proc.h`.

Η δομή στο `include/proc.h`

```
93  struct proc
94  {
95    uintp sz;                     // Size of process memory (bytes)
96    pde_t *pgdir;                 // Page table
97    char *kstack;                 // Bottom of kernel stack for this process
98    enum procstate state;         // Process state
99    volatile int pid;             // Process ID
100   struct proc *parent;          // Parent process
101   struct trapframe *tf;         // Trap frame for current syscall
102   struct context *context;      // swtch() here to run process
103   void *chan;                   // If non-zero, sleeping on chan
104   int killed;                   // If non-zero, have been killed
105   struct file *ofile[NOFILE]; // Open files
106   struct inode *cwd;            // Current directory
107   char name[16];                // Process name (debugging)
108
109   int inuse;   // If it's being run by a CPU or not
110   int ticks;   // How many ticks has accumulated
111   int tickets; // Number of tickets this process has (for lottery scheduling)
112 };
```

Μνήμη και αποθηκευμένοι καταχωρητές

Πεδίο	Τι κρατάει το kernel
`sz`	Το μέγεθος της μνήμης χρήστη σε bytes. Ο τύπος `uintp` είναι ακέραιος με το πλάτος ενός δείκτη.
`pgdir`	Δείκτη στον πίνακα σελίδων. Μέσω αυτού η CPU μεταφράζει τις εικονικές διευθύνσεις της διεργασίας σε φυσικές.
`kstack`	Τη βάση της στοίβας που χρησιμοποιεί η διεργασία όταν εκτελεί κώδικα του πυρήνα.
`tf`	Δείκτη στο trap frame. Εκεί αποθηκεύονται οι καταχωρητές και η διεύθυνση επιστροφής όταν, για παράδειγμα, ένα system call μάς φέρει από το user space στον πυρήνα.
`context`	Δείκτη στο αποθηκευμένο kernel context. Από εδώ η `swtch()` βρίσκει τη στοίβα για να συνεχιστεί η εκτέλεση μέσα στον πυρήνα.

Κατάσταση, πόροι και στατιστικά της διεργασίας

Πεδίο	Τι κρατάει το kernel
`state`	Την κατάσταση της θέσης ή της διεργασίας. Μια θέση ξεκινά ως `UNUSED` και δεσμεύεται ως `EMBRYO`. Η διεργασία είναι `RUNNABLE` όσο περιμένει CPU, `RUNNING` όσο εκτελείται και `SLEEPING` όσο περιμένει κάποιο γεγονός. Με την `exit()` μένει `ZOMBIE` ώσπου η `wait()` να ελευθερώσει τη θέση.
`pid`	Τον αριθμό που αναγνωρίζει τη συγκεκριμένη διεργασία.
`parent`	Δείκτη στην εγγραφή του γονέα. Η `wait()` τον ελέγχει όταν ψάχνει ποιες διεργασίες είναι παιδιά της καλούσας.
`chan`	Το κανάλι αναμονής της `sleep()`. Η `wakeup(chan)` κάνει `RUNNABLE` όσες διεργασίες κοιμούνται στο ίδιο κανάλι.
`killed`	Σημαία που θέτει η `kill()`. το kernel την ελέγχει πριν επιστρέψει στο user space και, αν έχει τεθεί, καλεί την `exit()`.
`ofile[NOFILE]`	Δείκτες στα ανοιχτά αρχεία. Ο file descriptor είναι ο αριθμός θέσης στον πίνακα, π.χ. το `ofile[1]` για την τυπική έξοδο.
`cwd`	Δείκτη στο inode του τρέχοντος καταλόγου, από όπου ξεκινά η αναζήτηση ενός σχετικού path.
`name[16]`	Το όνομα που εμφανίζει η `procdump()` για το debugging.
`inuse`	Γίνεται 1 στην πρώτη εκτέλεση και 0 στην `exit()`. Παραμένει 1 και στον ύπνο. Για να δούμε αν τρέχει τώρα, ελέγχουμε το `state`.
`ticks`	Timer ticks σε user mode (μόνο στη CPU 0).
`tickets`	Τα εισιτήρια της διεργασίας. Ο αρχικός scheduler αγνοεί το πεδίο. Στην Άσκηση 3 τα χρησιμοποιούμε για την κλήρωση.

2 Άσκηση 2: Οι συναρτήσεις `scheduler()` και `sched()`

1. Πώς παίρνει τη CPU μια διεργασία

Κάθε CPU τρέχει τη δική της `scheduler()` στο `kernel/proc.c`. Στην έκδοση πριν από την Άσκηση 3, η συνάρτηση σαρώνει επανειλημμένα τον `ptable.proc` με τη σειρά. Ενεργοποιεί τα interrupts με `sti()` και παίρνει το `ptable.lock` πριν εξετάσει τις εγγραφές. Έτσι, δύο CPU δεν μπορούν να επιλέξουν ταυτόχρονα την ίδια `RUNNABLE` διεργασία.

```
265 void scheduler(void)
266 {
267   struct proc *p = 0;
268
269   for (;;)
270   {
271     // Enable interrupts on this processor.
272     sti();
273
274     // no runnable processes? (did we hit the end of the table last time?)
275     // if so, wait for irq before trying again.
276     if (p == &ptable.proc[NPROC])
277       hlt();
278
279     // Loop over process table looking for process to run.
280     acquire(&ptable.lock);
281     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
282     {
283       if (p->state != RUNNABLE)
284         continue;
285
286       // Switch to chosen process.  It is the process's job
287       // to release ptable.lock and then reacquire it
288       // before jumping back to us.
289       proc = p;
290       switchuvm(p);
291       p->state = RUNNING;
292       p->inuse = 1;
293
294       swtch(&cpu->scheduler, proc->context);
295
296       switchkvm();
297       // Process is done running for now.
298       // It should have changed its p->state before coming back.
299       proc = 0;
300     }
301     release(&ptable.lock);
302   }
303 }
```

Ας πάρουμε την περίπτωση που ο scheduler βρίσκει το shell έτοιμο. Βάζει τον δείκτη του στο `proc`, που είναι τοπικό στη συγκεκριμένη CPU, και η `switchuvm(p)` φορτώνει τον πίνακα σελίδων του shell. Στη γραμμή 294 η `swtch()` αποθηκεύει το context του scheduler στο `cpu->scheduler` και φορτώνει εκείνο του shell. Από εδώ και πέρα η CPU εκτελεί κώδικα της διεργασίας.

Η `switchkvm()` στη γραμμή 296 εκτελείται όταν ο έλεγχος γυρίσει στον scheduler. Επαναφέρει τον πίνακα σελίδων του πυρήνα. Το `proc = 0` δηλώνει ότι η συγκεκριμένη CPU βρίσκεται πάλι στον scheduler. Η εγγραφή του shell παραμένει στον `ptable.proc` με την κατάσταση που έθεσε πριν παραδώσει τη CPU. Ο αρχικός scheduler συνεχίζει τη σάρωση από την επόμενη θέση. Στο τέλος του πίνακα αφήνει το lock και περιμένει το επόμενο interrupt με `hlt()`.

2. Πώς επιστρέφει ο έλεγχος στον scheduler

Η τρέχουσα διεργασία καλεί τη `sched()` μέσα από τον πυρήνα. Για παράδειγμα, όταν λήξει το κβάντο της, ο timer interrupt οδηγεί στη `yield()`. Αυτή παίρνει το `ptable.lock`, θέτει `state = RUNNABLE` και καλεί τη `sched()`. Το ίδιο σημείο χρησιμοποιούν η `sleep()` αφού θέσει `SLEEPING` και η `exit()` αφού θέσει `ZOMBIE`.

```
307 void sched(void)
308 {
309   int intena;
310
311   if (!holding(&ptable.lock))
312     panic("sched ptable.lock");
313   if (cpu->ncli != 1)
314     panic("sched locks");
315   if (proc->state == RUNNING)
316     panic("sched running");
317   if (readeflags() & FL_IF)
318     panic("sched interruptible");
319   intena = cpu->intena;
320   swtch(&proc->context, cpu->scheduler);
321   cpu->intena = intena;
322 }
```

Πριν από την αλλαγή, η `sched()` ελέγχει ότι κρατάμε μόνο το `ptable.lock` (`ncli == 1`), ότι τα interrupts είναι απενεργοποιημένα και ότι η διεργασία έχει ήδη φύγει από το `RUNNING`. Στη γραμμή 320 η `swtch()` σώζει καταχωρητές στη στοίβα πυρήνα της διεργασίας, γράφει τη θέση της στοίβας στο `proc->context` και φορτώνει τη στοίβα του scheduler. Το `ret` συνεχίζει από τη διεύθυνση επιστροφής που βρίσκεται σε εκείνη τη στοίβα.

Αν το shell είχε καλέσει `yield()`, την επόμενη φορά που θα επιλεγεί συνεχίζει ακριβώς μετά τη δική του `swtch()`. Η `sched()` επαναφέρει το `intena`, δηλαδή την αποθηκευμένη ένδειξη για την επανενεργοποίηση των interrupts, και η `yield()` αφήνει το lock.

3. Η υλοποίηση της `swtch()`

Ο πηγαίος κώδικας του `kernel/swtch.S` δείχνει το πραγματικό πέρασμα από τη μία στοίβα πυρήνα στην άλλη. Τα `pushl` αποθηκεύουν τους καταχωρητές στην παλιά στοίβα, το `movl` αλλάζει τον `esp`, και τα `popl` μαζί με το `ret` συνεχίζουν από το αποθηκευμένο context. Αυτή είναι η παραλλαγή 32-bit· το προεπιλεγμένο 64-bit build χρησιμοποιεί το αντίστοιχο `kernel/swtch64.S`.

```
1  # Context switch
2  #
3  #   void swtch(struct context **old, struct context *new);
4  #
5  # Save current register context in old
6  # and then load register context from new.
7
8  .globl swtch
9  swtch:
10   movl 4(%esp), %eax
11   movl 8(%esp), %edx
12
13   # Save old callee-save registers
14   pushl %ebp
15   pushl %ebx
16   pushl %esi
17   pushl %edi
18
19   # Switch stacks
20   movl %esp, (%eax)
21   movl %edx, %esp
22
23   # Load new callee-save registers
24   popl %edi
25   popl %esi
26   popl %ebx
27   popl %ebp
28   ret
```

3 Άσκηση 3: Lottery Scheduling στο xv6

Με τη νέα `scheduler()` κάθε `RUNNABLE` διεργασία συμμετέχει στην κλήρωση με τα δικά της εισιτήρια. Ας πάρουμε τρεις διεργασίες με 1, 2 και 3 tickets. Κληρώνουμε έναν αριθμό από το 0 έως το 5. Το 0 ανήκει στην πρώτη, τα 1 και 2 στη δεύτερη και τα 3, 4 και 5 στην τρίτη. Άρα η τρίτη έχει πιθανότητα $3/6$ να πάρει την επόμενη σειρά στη CPU. Αν κοιμηθεί, μένουν τρία εισιτήρια και η πιθανότητα της πρώτης ανεβαίνει από $1/6$ σε $1/3$.

1. Αρχικά tickets, κληρονόμηση και `settickets()`

Στο `kernel/proc.c`, η `allocproc()` δίνει ένα εισιτήριο σε κάθε νέα θέση. Έτσι και η `init` μπορεί να κερδίσει την πρώτη κλήρωση.

```
42 found:
43   p->state = EMBRYO;
44   p->pid = nextpid++;
45   p->ticks = 0;
46   p->tickets = 1;
47   release(&ptable.lock);
```

Στη `fork()` αντιγράφουμε τα tickets του γονέα πριν γίνει το παιδί `RUNNABLE`.

```
154   pid = np->pid;
155   safestrcpy(np->name, proc->name, sizeof(proc->name));
156   acquire(&ptable.lock);
157   np->tickets = proc->tickets;
158   np->state = RUNNABLE;
159   release(&ptable.lock);
160   return pid;
```

Η δήλωση στο `include/user.h`, ο αριθμός της κλήσης, το assembly wrapper και η εγγραφή στον πίνακα `syscalls[]` υπήρχαν ήδη. Αφαιρέσαμε από το `kernel/syscall.c` την προσωρινή `sys_settickets()` που επέστρεφε πάντα -1. Στο `kernel/sysproc.c` η υλοποίηση ελέγχει το όρισμα και αλλάζει μόνο τα tickets της διεργασίας που έκανε την κλήση. Το 0 δηλώνει επιτυχία.

```
38 int sys_settickets(void)
39 {
40   int n;
41   if (argint(0, &n) < 0 || n < 1)
42     return -1;
43   acquire(&ptable.lock);
44   proc->tickets = n;
45   release(&ptable.lock);
46   return 0;
47 }
```

2. Η κλήρωση στον `kernel/proc.c`

Αθροίζουμε τα εισιτήρια των `RUNNABLE` διεργασιών. Η `draw_ticket()` κρατά όσα bits χρειάζονται και ξανατραβά αριθμό αν αυτός πέσει έξω από το διάστημα. Για έξι εισιτήρια, για παράδειγμα, η μάσκα κρατά τρία bits και οι τιμές 6 και 7 απορρίπτονται. Στο δεύτερο πέρασμα αφαιρούμε τα εισιτήρια κάθε διεργασίας ώσπου ο αριθμός να πέσει στο διάστημα του νικητή. Το `ptable.lock` καλύπτει και τα δύο περάσματα. Όταν η διεργασία επιστρέψει στον scheduler, το `break` οδηγεί σε καινούρια κλήρωση.

```
281 void scheduler(void)
282 {
283   struct proc *p;
284 
285   for (;;) {
286     sti();
287     acquire(&ptable.lock);
288     unsigned long long total = 0;
289     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
290       if (p->state == RUNNABLE)
291         total += p->tickets;
292 
293     if (total == 0) {
294       release(&ptable.lock);
295       hlt();
296       continue;
297     }
298 
299     unsigned long long ticket = draw_ticket(total);
300     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
301       if (p->state != RUNNABLE)
302         continue;
303       if (ticket >= p->tickets) {
304         ticket -= p->tickets;
305         continue;
306       }
307 
308       // Switch to chosen process.  It is the process's job
309       // to release ptable.lock and then reacquire it
310       // before jumping back to us.
311       proc = p;
312       switchuvm(p);
313       p->state = RUNNING;
314       p->inuse = 1;
315 
316       swtch(&cpu->scheduler, proc->context);
317 
318       switchkvm();
319       // Process is done running for now.
320       // It should have changed its p->state before coming back.
321       proc = 0;
322       break; // Draw again for the next dispatch.
323     }
324     release(&ptable.lock);
325   }
326 }
```

3. Γεννήτρια

Η εκφώνηση αναφέρει το `include/random.h`. Στο δικό μας xv6 η `rand()` βρίσκεται στο τέλος του `kernel/proc.c`. Την επεκτείναμε ώστε να επιστρέφει 64-bit αριθμό: το άθροισμα των tickets μπορεί να υπερβεί τα 2^{32}, παρότι κάθε διεργασία έχει 32-bit πλήθος. Ο scheduler την καλεί μέσω της `draw_ticket()` όσο κρατά το `ptable.lock`.

4. Κώδικας του `user/schedtest.c`

Ο γονέας δημιουργεί τρία παιδιά που εκτελούν ατέρμονα βρόχο. Διαβάζει τους μετρητές τους πριν και μετά από 600 ticks, τυπώνει τη διαφορά και τα τερματίζει. Η δοκιμή με $2^{31} - 1$ tickets ανά παιδί βασίζεται στη νέα 64-bit `rand()`, ώστε η κλήρωση να καλύπτει όλο το άθροισμα των tickets.

```c
#include "types.h"
#include "user.h"
#include "pstat.h"
int main(int argc, char **argv)
{
  int tickets[3] = {1, 2, 3}, pid[3], start[3], count = 0;
  int i, j;
  struct pstat p;

  if (argc != 1 && argc != 4) {
    printf(2, "usage: schedtest [tickets1 tickets2 tickets3]\n");
    exit();
  }
  for (i = 1; i < argc; i++) {
    tickets[i - 1] = atoi(argv[i]);
    if (tickets[i - 1] <= 0) {
      printf(2, "tickets must be positive\n");
      exit();
    }
  }
  for (i = 0; i < 3; i++) {
    if (settickets(tickets[i]) < 0 || (pid[i] = fork()) < 0)
      break;
    if (pid[i] == 0)
      for (;;)
        ; // Just wait
    count++;
  }
  if (count != 3 || getpinfo(&p) < 0)
    goto done;
  for (i = 0; i < 3; i++) {
    for (j = 0; j < NPROC && p.pid[j] != pid[i]; j++)
      ;
    if (j == NPROC)
      goto done;
    start[i] = p.ticks[j];
  }
  sleep(600);
  if (getpinfo(&p) < 0)
    goto done;
  printf(1, "PID TICKETS TICKS\n");
  for (i = 0; i < 3; i++) {
    for (j = 0; j < NPROC && p.pid[j] != pid[i]; j++)
      ;
    if (j < NPROC)
      printf(1, "%d %d %d\n", pid[i], tickets[i], p.ticks[j] - start[i]);
  }
done:
  for (i = 0; i < count; i++)
    kill(pid[i]);
  for (i = 0; i < count; i++)
    wait();
  exit();
}
```

5. Δοκιμή με τρεις διεργασίες σε μία CPU

Ξεκινάμε το QEMU με `make CPUS=1 qemu-nox`. Στο shell του xv6 δίνουμε, για παράδειγμα, `schedtest 1 2 3`. Πριν από κάθε `fork()`, ο γονέας καλεί τη `settickets(n)`. Με τη `getpinfo()` διαβάζει τους αρχικούς μετρητές, κοιμάται για 600 ticks και τους ξαναδιαβάζει. Η διαφορά δίνει τον χρόνο CPU που πήρε κάθε παιδί στο ίδιο διάστημα.

Tickets ανά παιδί	Ticks 1ου	Ticks 2ου	Ticks 3ου
$1,\ 1,\ 1$	197	209	194
$1,\ 2,\ 3$	103	196	301
$3,\ 2,\ 1$	285	194	121
$2^{31}-1$ στο καθένα	195	205	203

Με ίσα tickets περιμένουμε περίπου 200 ticks ανά παιδί. Με την αναλογία $1:2:3$ περιμένουμε 100, 200 και 300. Το τρίτο παιδί πήρε 301 από τα 600 ticks ($50{,}2\%$). Όταν αντιστρέψαμε τα tickets, το πρώτο παιδί πήρε τα περισσότερα (285). Οι μικρές αποκλίσεις είναι αναμενόμενες, αφού σε κάθε επιλογή γίνεται νέα κλήρωση. Η τελευταία δοκιμή ελέγχει και το άθροισμα. Τα τρία παιδιά έχουν μαζί $6\,442\,450\,941$ tickets, οπότε χρειαζόμαστε περισσότερα από 32 bits.

Αν δώσουμε `schedtest 0 1 2`, το πρόγραμμα απορρίπτει το μη θετικό όρισμα πριν από τη `fork()`. Το `forktest` ολοκληρώθηκε με `fork test OK`.